Computer Programming 1

Introduction to the course

(11-09-2026)

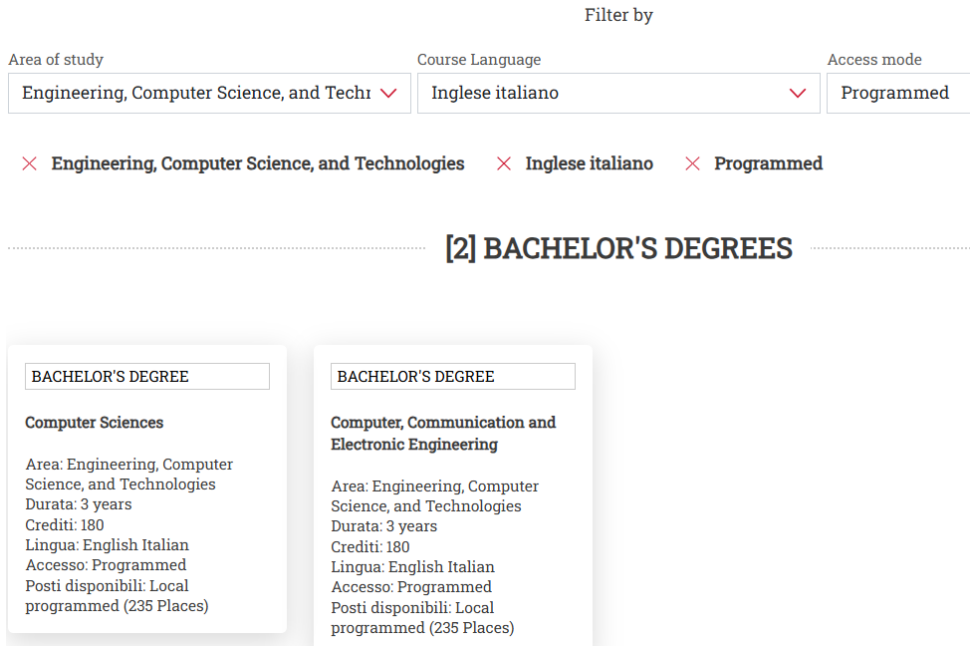
Welcome!
… *the university*

- University of Trento
- Department of Information Engineering and Computer Science (DISI)
- Undergraduate Courses - curriculum in English
  - Computer Sciences
  - Computer, Communications and Electronic Engineering (ICE)
- Academic Year: 2026-2027



Welcome! … *the course*

- Course: Computer Programming 1 (ENG)
  - Compulsory course
  - Schedule:
    - Period: first semester of 2026 – 2027
    - Lesson period: September – December 2026
    - Exams: Jan. & Feb. 2027, Jun. & Jul. 2027, Sept. 2027
  - Academic hours:
    - lecture (~45%)
    - laboratory (~55%)
  - (Course&Exam) Modality: in-person
  - 12 credits (ECTS)

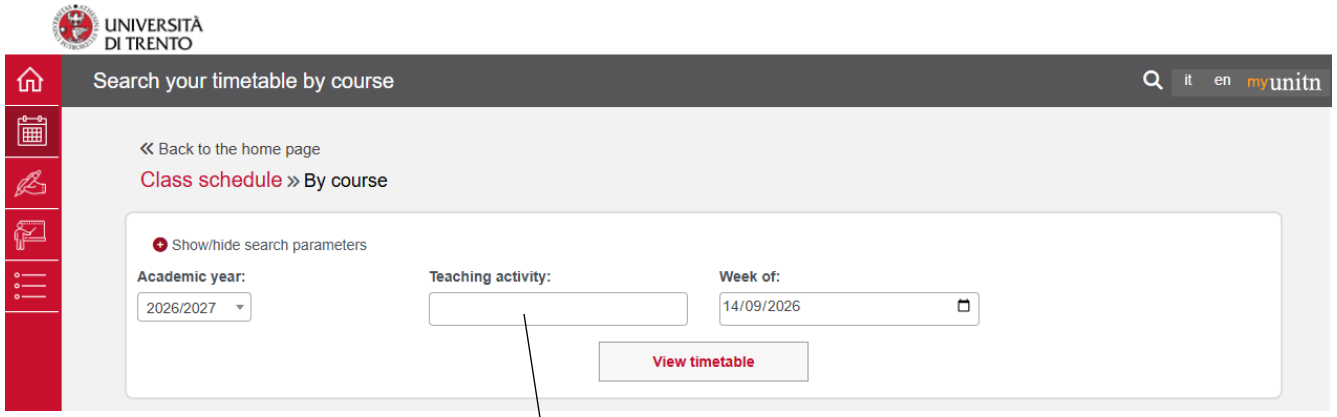


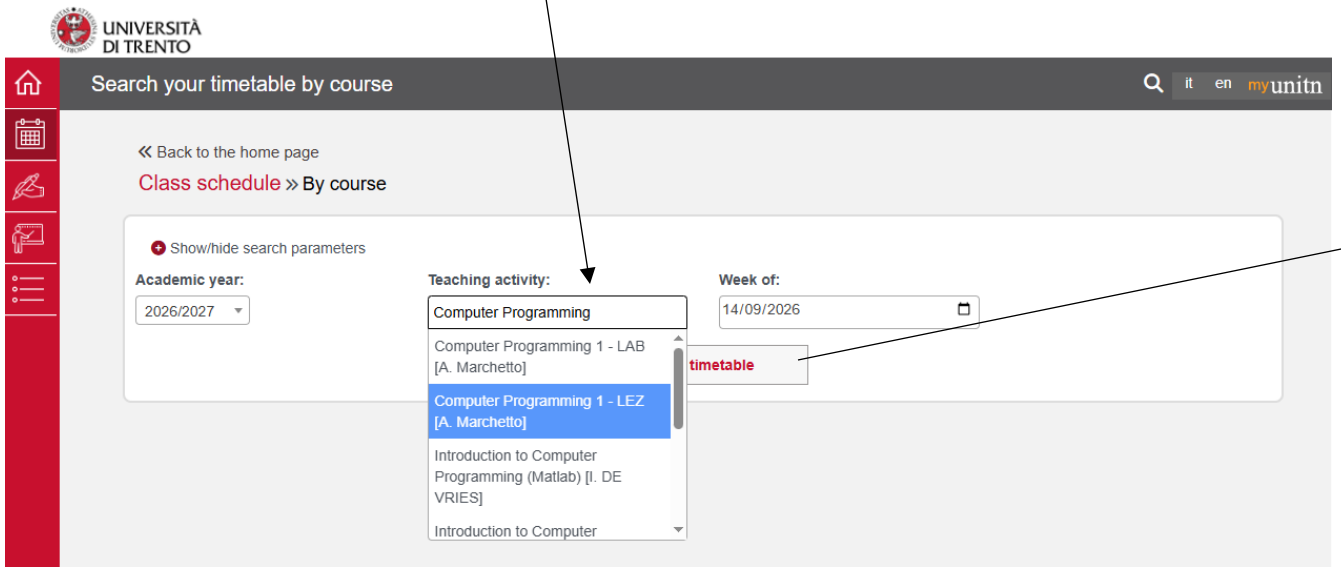
Credits: 180
➔ Courses: 12 or 6 credits
➔ Credits for other activities: thesis, etc

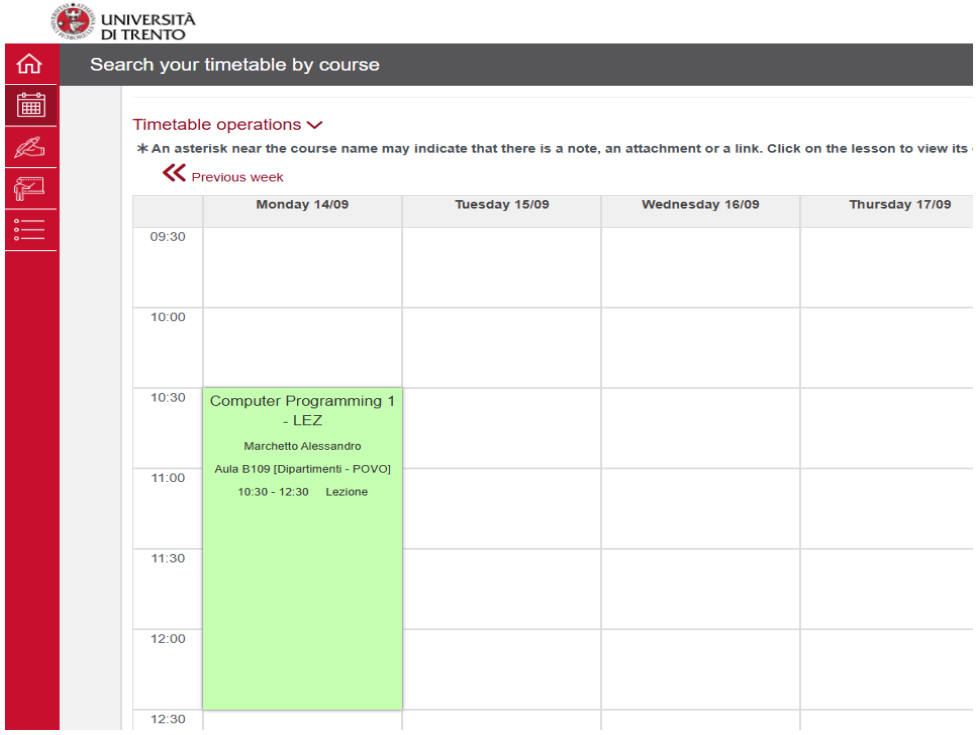
12 CFU x 25 hours/CFU = 300 hours
- Lectures, classes, and in-class activities
- Individual study
- Other activities, if required

Course schedule

Link to the Course Schedule







Course schedule

Day	Time	Location and room
Monday	10.30 – 12.30	Povo 1, room B109
Tuesday	12.30 – 14.30	Povo 2, room B106 pc
Wednesday	10.30 – 13.30	Povo 2, room B106 pc
Friday	13.30 – 15.30	Povo 2, room B107
Tuesday	8.30 – 10.30	Povo 1, room A201 pc

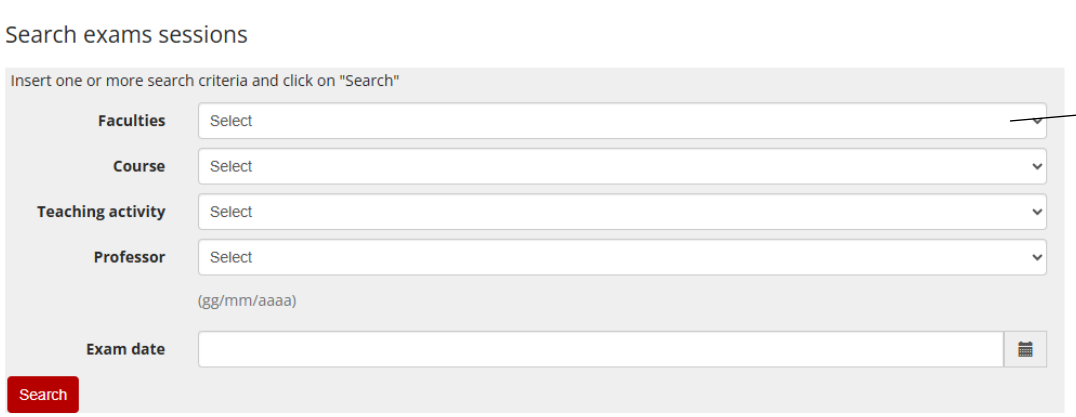
Lecturers

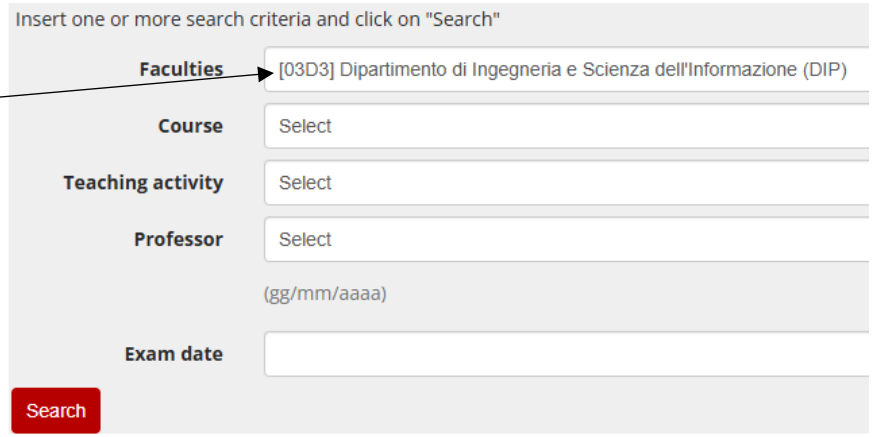
Laboratories

Tutoring
(from Sept.22, 2026)

Unitn Esse3







Link to the UniTN Esse3

- https://www.esse3.unitn.it

- *Check the information about the exams, e.g., days, rooms (when they will be published)*
- *Register for the exam session you plan to attend*

Unitn Moodle

Moodle platform of the course

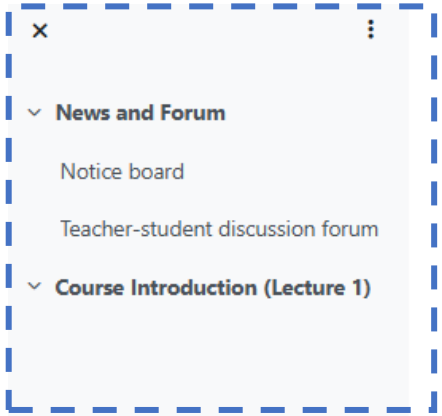
- https://didatticaonline.unitn.it/dol/course/view.php?id=44079

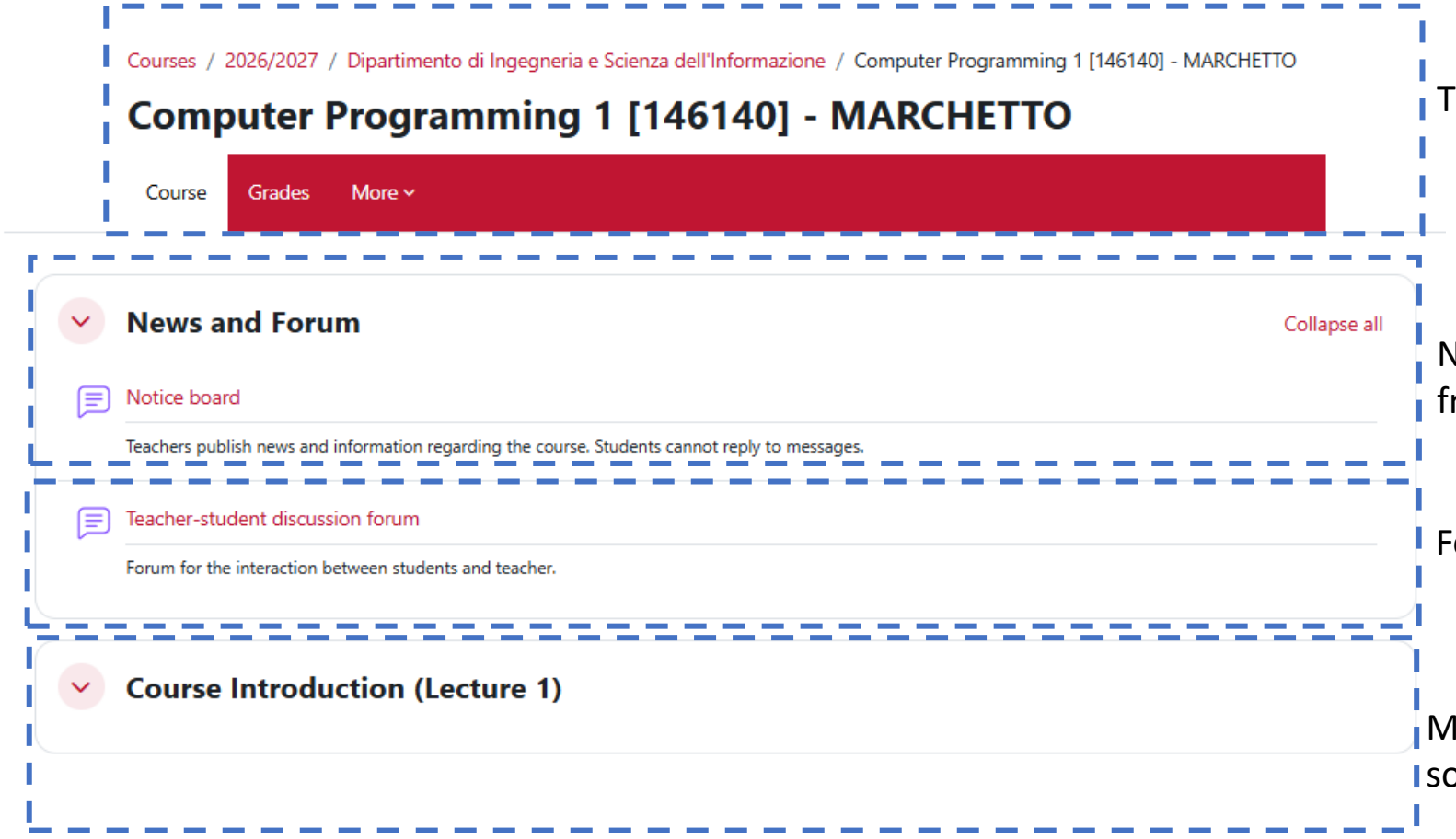
- *News and communication about the course and lectures*
- *Course material (slides, exercises, source code)*

- *Remember **to register** for the course on the Moodle platform!*

Unitn Moodle

Sections





The course

News and messages from teachers

Forum

Material (Slides, source code, etc.)

Who we are (1/3)

Alessandro Marchetto

- Associate Professor at UniTN
- ~10y experience in industry
- ~10y as a researcher in academia & research centers
- Ph.D. Computer science









Who we are (2/3)

Adolfo Villafiorita



- >10y Head of Research Unit at FBK
- >10y contract professor in Universities
- PhD Computer Science

Michele Bof



- Master's Degree in Computer Science @UniTN
- Senior developer

Co-founder of Shair.Tech, a startup developing technologies for reducing (food) waste and helping to make our distribution chain fairer, more sustainable, and circular.

Who we are (3/3)

Tutors

- Giuseppe Reina

  Bachelor degree, Computer, Communication and Electronic Engineering

- Enrico Dalla Croce

  Master degree, Computer Science

- Lorenzo Carlin

  Bachelor degree, Computer Science

Tutoring activities
Tutoring (without the teachers) *(from Sept.22, 2026)* Tuesday 8.30 – 10.30 Povo 1, room A201 pc
Tutoring during all laboratories Tuesday 12.30 – 14.30 Wednesday 10.30 – 13.30 Povo 2, room B106 pc

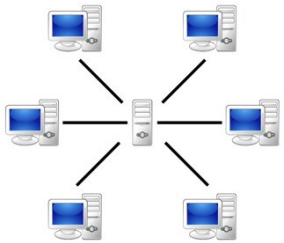
Context: Computer and Communications

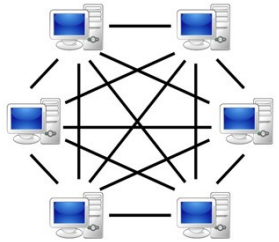






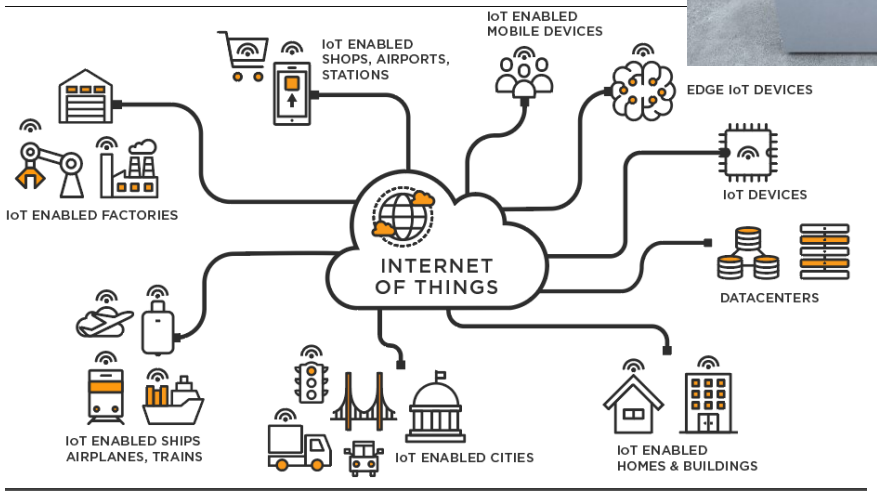


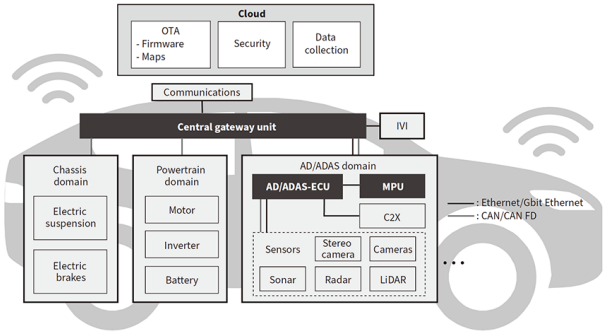














Informatics and Computer Science (1)

In general, these terms are used as synonyms even if they are not the same

Informatics (*Information processing science*)

- It is the study of computation, information, and automation
- It studies the representation, processing, and communication of information in natural and engineered systems
- It involves several scientific and technical aspects concerning the collection and processing of information in the various scientific, technological, and social sectors and activities

Computer Science

- It mainly refers to the application to engineering, science, law, medicine, …
- It involves theoretical disciplines: algorithms, information theory, …
- It is applied to practical disciplines: the design and implementation of software
- It is closely related to computer programming

Informatics and Computer Science (2)

Example: software engineering

Informatics

- Design and optimization of the information flow of a system (e.g., management of a customer care), by considering different user roles and their responsibility

Computer Science

- Develop the platform underlying a system (e.g., management of a customer care)
- Definition of an efficient ordering algorithm (for reducing time and complexity)
- Optimization of language compilator for speed up the program execution

Course objective

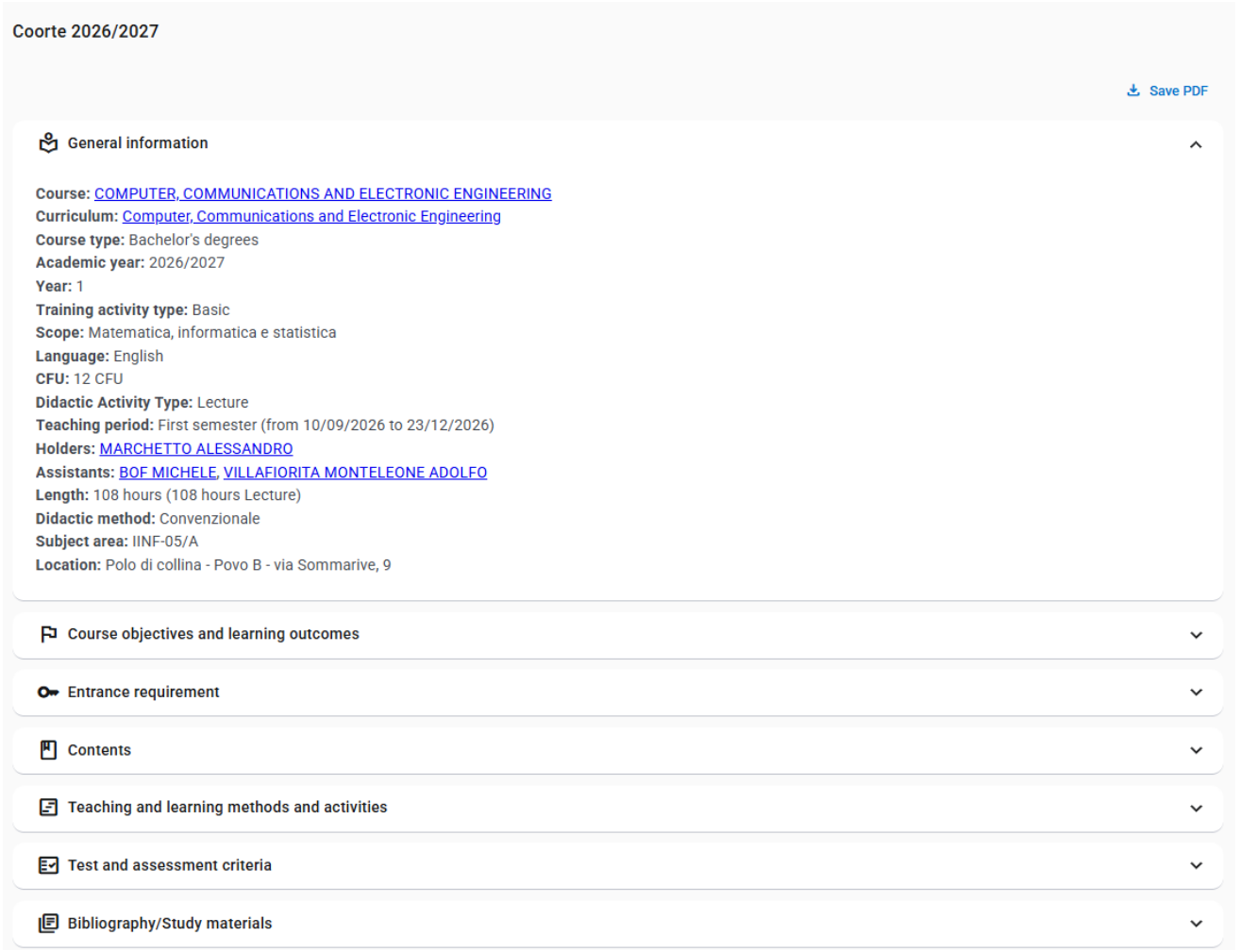
The main objective of the course is *to acquire basic knowledge of*:

- Computer
- Algorithms
- Programming language (one in particular)
- Programming

Syllabus

Introducing the course:

- Course objectives and learning outcomes
- Entrance requirement
- Contents
- Teaching and learning methods and activities
- Test and assessment criteria
- Bibliography/Study material



Coorte 2026/2027

Save PDF

General information

Course: COMPUTER, COMMUNICATIONS AND ELECTRONIC ENGINEERING
Curriculum: Computer, Communications and Electronic Engineering
Course type: Bachelor's degrees
Academic year: 2026/2027
Year: 1
Training activity type: Basic
Scope: Matematica, informatica e statistica
Language: English
CFU: 12 CFU
Didactic Activity Type: Lecture
Teaching period: First semester (from 10/09/2026 to 23/12/2026)
Holders: MARCHETTO ALESSANDRO
Assistants: BOF MICHELE, VILLAFIORITA MONTELEONE ADOLFO
Length: 108 hours (108 hours Lecture)
Didactic method: Convenzionale
Subject area: IINF-05/A
Location: Polo di collina - Povo B - via Sommarive, 9

Course objectives and learning outcomes

Entrance requirement

Contents

Teaching and learning methods and activities

Test and assessment criteria

Bibliography/Study materials

Link to the Course Syllabus

Course Objectives and Learning Outcomes

Introductory class on programming, programming principles and techniques

Learning Objectives

- Introduce algorithms, programming principles and techniques.
- Familiarize with code programming and provide the basic notions, insights and tools of code programming
- Learn fundamentals of C++ imperative programming
- Learn introductory notions of object-oriented programming in C++
- Learn program structures, elements, and operations
- Learn by doing: learn how to develop programs to solve problems of simple/average difficulty

Expected outcomes

The student:

(1) will know the fundamental notions and concepts about (C++) imperative programming;

(2) will be able to analyze problems of simple/average difficulty;

(3) will be able to identify and code solutions to problems of average difficulty by using C++ as imperative programming language.

Course Objectives and Learning Outcomes: FAQs

Is this a course on advanced programming? NO

Is this a course to learn how to program network / mobile / web apps? NO

Can I develop the programs in my own way ? NO

Can I use my personal devices, IDEs, tools, programming language, style? NO

So what?

- This is an introductory class on programming
- The main objective is to provide all the knowledge to model, solve and develop software program for simple problems.

Coding is not programming (1)

Coding
Computers can understand machine code that is difficult to understand for humans. Coding is the process of writing code in a programming language so that the computers can understand what it needs to execute. Coding is writing lines of code in the programming language to create a computer program (software).

Coding is not programming (1)

Coding

Computers can understand machine code that is difficult to understand for humans.
Coding is the process of writing code in a programming language so that the computers can understand what it needs to execute.
Coding is writing lines of code in the programming language to create a computer program (software).

Programming

Coding is only a phase of a software development.
Several phases are needed: *planning, design, testing, deployment, maintenance*.
Programming is a complex activity which includes not only coding, but also other activities, e.g., to design, analyze and implement algorithms, understand data structures, and solve problems.
To write code, you need to be clear about structure of the program: e.g., write the pseudocode to describe the logic of the algorithm to be implemented and use it for explaining the algorithm to a programmer (coder).

Coding is not programming (2)

	Coding	Programming

If programming is the process of writing a whole book.
Then coding is just about writing a single chapter of the book.

https://hackr.io/blog/coding-vs-programming-difference-you-should-know

Coding is not programming (2)

	Coding	Programming
Scope	It is a process in which a set of instructions are converted into a language that is comprehensible for a computer	Beyond coding, it involves the definition of the requirements, problem solving, pseudocode writing, algorithm thinking, optimizing, testing, executable code creation, maintenance

If programming is the process of writing a whole book.
Then coding is just about writing a single chapter of the book.

https://hackr.io/blog/coding-vs-programming-difference-you-should-know

Coding is not programming (2)

	Coding	Programming
Scope	It is a process in which a set of instructions are converted into a language that is comprehensible for a computer	Beyond coding, it involves the definition of the requirements, problem solving, pseudocode writing, **algorithm thinking**, optimizing, testing, executable code creation, maintenance
Skill	As a coder, a good knowledge of syntax and semantics of the selected programming language is required	As a programmer, additional skills are required compared to coding, e.g., high-level thinking capability and of problem analysis

If Programming is the process of writing a whole book.
Then Coding is just about writing a single chapter of the book.

https://hackr.io/blog/coding-vs-programming-difference-you-should-know

Course Content (not ordered)

Content by topics

Course Content (not ordered)

Content by topics

Basic code programming concepts

- Algorithms and PC architectures
- Programming phases and languages
- Introduction to (imperative) C++

Course Contents (not ordered)

Content by topics
Basic code programming concepts • Algorithms and PC architectures • Programming phases and languages • Introduction to (imperative) C++
Variables and predefined types • Representation, variables and constants • Data type (integers, booleans, characters)

Course Contents (not ordered)

Content by topics
Basic code programming concepts • Algorithms and PC architectures • Programming phases and languages • Introduction to (imperative) C++
Variables and predefined types • Representation, variables and constants • Data type (integers, booleans, characters)
Control structure • Basic instructions • Expressions and assignments • Structured instructions (sequences, conditions, cycles)

Course Contents (not ordered)

Content by topics
Basic code programming concepts • Algorithms and PC architectures • Programming phases and languages • Introduction to (imperative) C++
Variables and predefined types • Representation, variables and constants • Data type (integers, booleans, characters)
Control structure • Basic instructions • Expressions and assignments • Structured instructions (sequences, conditions, cycles)
Input/Output • I/O standard • I/O on files (argc & argv)

Course Contents (not ordered)

Content by topics

Basic code programming concepts

- Algorithms and PC architectures
- Programming phases and languages
- Introduction to (imperative) C++

Variables and predefined types

- Representation, variables and constants
- Data type (integers, booleans, characters)

Control structure

- Basic instructions
- Expressions and assignments
- Structured instructions (sequences, conditions, cycles)

Input/Output

- I/O standard
- I/O on files (argc & argv)

Advanced data types and memory management

- Array
- Ordered array
- Array multidimensional (matrices)
- Strings and texts
- Pointers and references
- Structure (struct)
- Dynamic memory allocation
- Dynamic allocation of array and struct
- Fundamental data structure (lists, stacks, and queues)
- Binary Trees and Graphs

Course Contents (not ordered)

Content by topics

Basic code programming concepts
- Algorithms and PC architectures
- Programming phases and languages
- Introduction to (imperative) C++

Variables and predefined types
- Representation, variables and constants
- Data type (integers, booleans, characters)

Control structure
- Basic instructions
- Expressions and assignments
- Structured instructions (sequences, conditions, cycles)

Input/Output
- I/O standard
- I/O on files (argc & argv)

Advanced data types and memory management
- Array
- Ordered array
- Array multidimensional (matrices)
- Strings and texts
- Pointers and references
- Structure (struct)
- Dynamic memory allocation
- Dynamic allocation of array and struct
- Fundamental data structure (lists, stacks, and queues)
- Binary Trees and Graphs

Program organization and structure
- Structure and organize a program on more files
- Functions and parameter passing
- Recursive functions
- Classes (introduction)

Course Contents (not ordered)

Content by topics	
Basic code programming concepts • Algorithms and PC architectures • Programming phases and languages • Introduction to (imperative) C++	**Advanced data types and memory management** • Array • Ordered array • Array multidimensional (matrices) • Strings and texts • Pointers and references • Structure (struct) • Dynamic memory allocation • Dynamic allocation of array and struct • Fundamental data structure (lists, stacks, and queues) • Binary Trees and Graphs
Variables and predefined types • Representation, variables and constants • Data type (integers, booleans, characters)	
Control structure • Basic instructions • Expressions and assignments • Structured instructions (sequences, conditions, cycles)	
Input/Output • I/O standard • I/O on files (argc & argv)	**Program organization and structure** • Structure and organize a program on more files • Functions and parameter passing • Recursive functions • Classes (introduction)
Advanced topics: program testing and debugging, generative programming ….(*depending on the time*)	

Entrance requirements

Prerequisites

- This is an introductory class on programming
- The objective is to provide all the knowledge to model, solve and develop software program for simple problems. Hence, there are no specific prerequisites to address the course.

Teaching and learning methods and activities

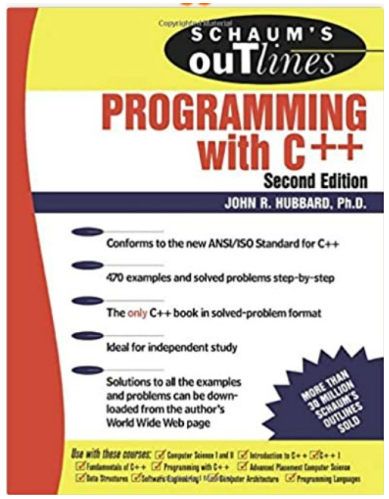
Teaching method

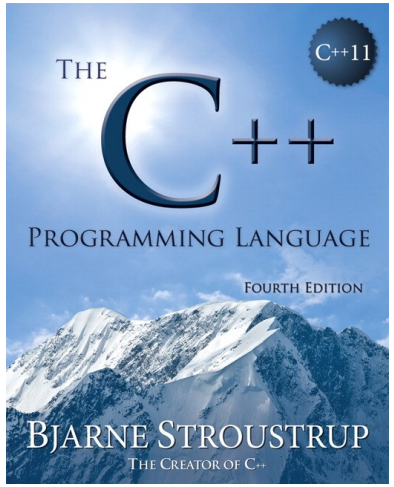
- Lectures for the theoretical part: introduction of notions, principles, tools, and examples of problems and programs
- Laboratory exercises mainly to practice with C++ programming (learning by doing)
- Tutoring Labs: tutoring sessions will be conducted in the laboratory to support the students while learning (tutoring by other students, and tutoring by teachers)

Bibliography/Study material

- Slides and additional documents will be available on the course web site
- Examples of code programs, the ones used in both lectures and laboratories, will be available on the course web site alongside additional examples
- Textbooks

1. Reference for programming with C++ (Alternatives)
   a) John R. Hubbard. “Schaum's Outline Of Programming With C++ (2ed)
      McGraw Hill, ISBN: 9780071353465
      https://www.mhprofessional.com/schaum-s-outline-of-programming-with-c-9780071353465-usa
   b) Bjarne Stroustrup “Programming: Principle and Practice Using C++”,
      Pearson - Addison-Wesley, ISBN: 9780321992789 (2nd ed.)
      https://www.pearson.com/uk/educators/higher-education-educators/program/Stroustrup-Programming-Principles-and-Practice-Using-C-2nd-Edition/PGM1087672.html?tab=overview
2. (optional) Reference for the C++ language: (Alternatives)
   a) Bjarne Stroustrup “The C++ Programming Language”
      Pearson – Addison-Wesley, ISBN: 9780321563842 (4 a ed.)
      https://www.pearson.com/uk/educators/higher-education-educators/program/Stroustrup-C-Programming-Language-The-4th-Edition/PGM948798.html
   b) Herbert Schildt “C++: The Complete Reference”
      McGraw Hill, ISBN: 9780071502399 (4 a ed.)
      https://www.mhprofessional.com/9780072226805-usa-c-the-complete-reference-4th-edition-group





Test and assessment criteria

Exam sessions:

- January 2027,
- February 2027,
- June 2027,
- July 2027,
- September 2027

Time: 2h each session

Each session is composed of two parts

- Questions (closed and open) to assess the acquired theoretical competence
- Exercises to assess the acquired practical competence in terms of C++ code programming

The exam will take place in the laboratory using the laboratory PC and tools (no other material is admitted)

Exam organization and grade

- Exam
  - The grade of the exam is expressed in /30 and it represents the weighted average of the scores of the two parts (40% theory and 60% laboratory)
  - Both parts are considered passed when a score of at least 18 is achieved.

- A simulation of the exam session will be organized (towards the end of the course).
  - Goal:
    1. understand how the exam is organized,
    2. self-evaluation
  - Not valid as an exam and not evaluated
  - To be scheduled (depending on the course): end of November 2026

Suggestions (1)

- Attend the lectures and the laboratories
- Please note that every topic discussed during the lectures and the laboratories could be part of the exam
- Asking question if you do not understand something is a good practice, for you and for the other course attendees
- Try to study day-by-day and after each lecture
- Try to execute/re-execute each exercise and examples that have been seen during the lectures
- Always try to solve the proposed exercises and … implement, implement, implement

- Experience plays a fundamental roles in programming
- Understanding the messages from the compiler and the debugger is crucial for a programmer, the experience helps in understanding them
- …

Suggestions (2)

- ...
- When programming:
  - Before starting to write the code, analyze the problem and organize (at least) a mental schema of the solution to implement
  - Write simple, clear and readable code as much as possible
    - Code indentation is strongly recommended!!!
    - Leave adequate spacing among code instructions
    - Use meaningful name and terms, by following the conventions (upper case, lower case, etc.)
    - ...
  - Adopt "coding standards":
    - Adopting coding standards leads to develop code with a quite clear and "uniform" format, this is important to decrease the code complexity and to increase the code readability, promoting code reusability, maintainability of the code, and identification of errors.
  - Be carefully
    - Deal with programming with humility … try to avoid: "I already know how to program"
    - If needed, be ready to change your way of working/programming: "I have always done it in this way!"

Interactions with teachers

- Ask questions before/during/after the lectures, laboratories, and breaks
- Discuss doubts and solutions with teachers and colleagues (outside the lecture/laboratory time!)
- Ask for 1-to-1 meetings only for clarifications and (personal) problems
- Ask questions via email: try to limit yourself to relevant questions, otherwise we will be not able to answer at all!
  - E.g.,
    - Meeting for clarifications
    - Alerts about mistakes and problems with the material
    - Alerts about problems such as time overlaps
    - Alerts about specific and personal problems of the student (e.g., working student, etc.)
- Tutoring will be activated *soon*
- Please, when contacting us, you have to use the official UniTN email: ***name.surname@studenti.unitn.it***

A note on plagiarism

Total or partial plagiarism is forbidden!

- Exam tests (written part)
- Exercises
- Programs
- …

- “I took this text from a colleague of mine” No! ... no reuse of material produced by other colleagues is allowed

- In case of plagiarism, you will fail the class.

Copyright and Credits

- The material (slide, text, figures, etc.) is intended solely for students at the University of Trento registered to the relevant course for the Academic Year 2026-2027.
- The material (slide, text, figures, etc.) used in the course is partially inspired to the one used in the past years by Marco Roveri, Giuseppe Riccardi, and Roberto Sebastiani.
- The copyright of the material is held by the authors. Copying, editing, translation, storage, processing or forwarding of content in databases or other electronic media and systems without written consent of the copyright holders is forbidden. The selling of (parts) of this material is forbidden. Presentation of the material to students not involved in the course is forbidden. The unauthorized reproduction or distribution of individual content or the entire material is not permitted and is punishable by law.